



Universitatea „Ștefan cel Mare” - Suceava
Facultatea de Inginerie Electrică și Știința
Calculatoarelor

PROIECT

SISTEM MONITORIZARE DEPOZIT

Grupa:
3122B

Student:
Valach Dumitru Cornel

Cuprins

CAPITOLUL 1: INTRODUCERE ȘI OBIECTIVELE SISTEMULUI	3
CAPITOLUL 2: ARHITECTURA SOFTWARE A APLICAȚIEI	3
CAPITOLUL 3: MODELUL RELAȚIONAL DE DATE (SQLITE3)	3
CAPITOLUL 4: IMPLEMENTAREA LOGICĂ ȘI POLIMORFISMUL	5
CAPITOLUL 5: ALGORITMUL AUTOMAT DE MENTENANȚĂ (CMMS).....	6
CAPITOLUL 6: MECANISMUL DE LIVE-REFRESH ÎN INTERFAȚA GRAPHICĂ.....	6
CAPITOLUL 7: CONCLUZII ȘI GHID DE COMPILARE.....	6

CAPITOLUL 1: INTRODUCERE ȘI OBIECTIVELE SISTEMULUI

Prezenta documentație descrie arhitectura, designul structural și detaliile de implementare ale unui sistem integrat de management al resurselor întreprinderii (Enterprise Resource Planning - ERP). Aplicația unifică trei componente software critice într-o singură soluție centralizată: un sistem de management al depozitului (Warehouse Management System - WMS), un sistem de management al transporturilor (Transport Management System - TMS) și un sistem computerizat de management al mentenanței (Computerized Maintenance Management System - CMMS).

Obiectivul principal al proiectului constă în automatizarea completă a fluxului logistic: de la recepția fizică a mărfurilor și clasificarea lor polimorfică, până la vânzarea, alocarea volumetrică pe vehicule de transport, monitorizarea încărcării flotei în timp real și gestionarea automată a uzurii flotei prin imobilizarea vehiculelor în vederea efectuării reviziilor tehnice obligatorii.

CAPITOLUL 2: ARHITECTURA SOFTWARE A APLICAȚIEI

Sistemul este construit respectând principiul separării responsabilităților (Separation of Concerns), fiind structurat pe trei niveluri arhitecturale distincte:

- **Nivelul de Prezentare (Frontend TUI):** Implementat utilizând biblioteca modernă FTXUI. Acesta randează componente grafice interactive în terminal (ferestre, tabele, meniuri și bare de progres) și interceptează evenimentele de la tastatură în mod reactiv.
- **Nivelul Logicii de Business (Core Engine):** Orchestrat de clasa Depozit. Aceasta acționează ca un mediator între starea din memorie (stocul stocat ca un tabel de dispersie) și nivelul de persistență, validând regulile operaționale.
- **Nivelul de Persistență (Gestiunea Datelor):** Încapsulat în clasa DatabaseManager, care interfațează direct cu motorul bazei de date relaționale SQLite3, izolând complet instrucțiunile SQL de restul aplicației.

CAPITOLUL 3: MODELUL RELAȚIONAL DE DATE (SQLITE3)

Persistența robustă este garantată de o bază de date SQLite3 locală structurată în 5 tabele

relaționale corelate. Schemele tabelelor reflectă evoluția dinamică a aplicației, incluzând suport pentru volume geometrice și contoare de uzură tehnică.

Tabelul: Produse

Camp	Tip Date	Constrângere
ID	INTEGER	PRIMARY KEY
Nume	TEXT	NOT NULL
Cantitate	INTEGER	DEFAULT 0
PragAlerta	INTEGER	DEFAULT 5
PretAchizitie	REAL	DEFAULT 0.0
PretVanzare	REAL	DEFAULT 0.0
Volum	REAL	DEFAULT 0.0

Tabelul: Camioane

Câmp	Tip Date	Constrângere
ID	TEXT	PRIMARY KEY (Ex: Număr Înmatriculare)
Capacitate	REAL	Exprimată în m3
Status	TEXT	'Disponibil', 'In Cursa', 'Necesita Service'
NumarCurse	INTEGER	DEFAULT 0 (Contor uzură tehnică)

Tabelul: IstoricService

Câmp	Tip Date	Constrângere
ID	INTEGER	PRIMARY KEY AUTOINCREMENT
ID_Camion	TEXT	FOREIGN KEY REFERENCES Camioane(ID)
DataRevizie	INTEGER	UNIX Timestamp
TipInterventie	TEXT	Descrierea procedurii efectuate

CAPITOLUL 4: IMPLEMENTAREA LOGICĂ ȘI POLIMORFISMUL

Gestiunea stocului utilizează principiile fundamentale ale programării orientate pe obiecte. Clasa de bază *Produs* încapsulează proprietățile geometrice și economice universale. Din aceasta sunt derivate clase specializate:

- *ProdusElectronic*: Adaugă atribute pentru tensiunea de alimentare și perioada de garanție exprimată în luni.
- *ProdusPerisabil*: Monitorizează temperatura optimă de depozitare și numărul de zile de valabilitate rămase.
- *ProdusFragil* / *ProdusVoluminos* / *ProdusPericulos*: Implementează restricții de stivuire, manipulare cu utilaje specifice (motostivuitoare) și clase de risc chimic/biologic.

Interfața grafică încarcă datele în mod dinamic prin mecanismul de paginare securizat la nivel de motor SQL, prevenind alocările masive de memorie RAM:

```
std::string sql = "SELECT ID, Nume, Cantitate, PragAlerta, PretAchizitie, PretVanzare,  
Volum FROM Produse ORDER BY ID ASC LIMIT " +  
std::to_string(limita) + " OFFSET " + std::to_string(offset) + ";;";
```

CAPITOLUL 5: ALGORITMUL AUTOMAT DE MENTENANȚĂ (CMMS)

Pentru a simula un mediu de transport industrial realist, clasa DatabaseManager controlează ciclul de viață al activelor logistice. Algoritmul implementează o mașină de stări finită (Finite State Machine) direct prin interogări tranzacționale native. Când un camion se întoarce la bază, o instrucțiune condiționată evaluează starea de uzură:

```
std::string sql = "UPDATE Camioane SET NumarCurse = NumarCurse + 1, "  
    "Status = CASE WHEN NumarCurse + 1 >= 3 THEN 'Necesita Service' ELSE  
'Disponibil' END "  
    "WHERE ID = ?";
```

Vehiculele blocate automat în statusul Necesita Service sunt imobilizate instantaneu în atelierul mecanic și eliminate complet din panourile de expediere de mărfuri. Deblocarea lor se poate face exclusiv prin apelarea procedurii de revizie, moment în care se resetează contorul și se generează o înregistrare inalterabilă în jurnalul de service, alături de data curentă computerizată.

CAPITOLUL 6: MECANISMUL DE LIVE-REFRESH ÎN INTERFAȚA GRAPHICĂ

O problemă majoră în aplicațiile TUI complexe o reprezintă desincronizarea datelor dintre panouri (Stale UI). În acest proiect, problema a fost rezolvată prin eliminarea stocării listelor statice în interiorul componentelor FTXUI.

Toate drop-down-urile și tabelele își reîncarcă listele vectoriale direct din baza de date SQLite în interiorul funcției lambda de randare (apelată la fiecare cadru / redesenare a ecranului). Astfel, în milisecunda în care utilizatorul confirmă o recepție sau o sosire de vehicul, datele devin imediat vizibile în toate celelalte tab-uri ale sistemului ERP fără a necesita repornirea sau trimiterea de semnale inter-procese.

CAPITOLUL 7: CONCLUZII ȘI GHID DE COMPILARE

Sistemul dezvoltat demonstrează cum un limbaj performant de nivel jos precum C++ poate fi

combinat cu o bază de date relațională serverless pentru a crea un instrument de management logistic rapid, sigur și complet automatizat.

Compilarea proiectului pe un mediu Linux curat se efectuează prin următoarea suită de comenzi în terminal:

```
# Generarea structurii de directoare prin CMake  
cmake -B build
```

```
# Compilarea nativă a executabilului binare  
cmake --build build
```

```
# Lansarea aplicației ERP în terminal  
./build/depozit_app
```